```
> restart;with(LinearAlgebra):with(Student):with(plots):
> interface(rtablesize=51);
```

10 **(1)**

```
> # First, you have to be sure that you are using the latest version
  of Physics Package, otherwise some commands may not work properly.
  In this work, we used the update Physics61.3 availabe on
  http://www.maplesoft.com/products/maple/features/physicsresearch.
  aspx
  Physics:-Version();
```

"/Library/Frameworks/Maple.framework/Versions/17/lib/Physics.mla", **(2)**

2014, March 19, 3:0 hours

```
> Mat:=proc(NS,NE,kp1,km1,k2)


  delta:=proc(a,b) piecewise(a=b,1,0); end proc;
  Baseset:=proc(sm,em,cm,pm) global B; global Bdim;
  i:=1;

  for si from 0 to sm
      do
         for ei from 0 to em
             do
                for ci from 0 to cm
                    do
                       for pri from 0 to pm
                           do
                           if ei+ci = em then
                           if si+ci+pri = sm then

                           B[i]:=(si,ei,ci,pri):
                           i:=i+1;
                           end if;
                           end if;
                           end do;
                    end do;
                end do;
           end do;
      Bdim:=i-1;
  end proc;
  Baseset(NS,NE,NE,NS):

  ME:=proc(si,sit,ei,eit,ci,cit,pri,prit,kp1,km1,k2) (-delta(ei, eit)
  *delta(ci, cit)*((k2+km1)*cit+kp1*sit*eit)*delta(si, sit)+kp1*sit*
  eit*delta(si, sit-1)*delta(ei, eit-1)*delta(ci, cit+1)+km1*cit*
  delta(si, sit+1)*delta(ei, eit+1)*delta(ci, cit-1))*delta(pri,
  prit)+k2*cit*delta(si, sit)*delta(ei, eit+1)*delta(ci, cit-1)*delta
  (pri, prit+1); end proc;

  element:= proc(i,j,kp1,km1,k2) ME(B[i][1],B[j][1],B[i][2],B[j][2],B
  [i][3],B[j][3],B[i][4],B[j][4],kp1,km1,k2); end proc;

     for i from 1 to Bdim do
         for j from 1 to Bdim do
         A[i,j]:= element(i,j,kp1,km1,k2);
```

```
        end do;
    end do;
Mt:= Array(1..Bdim,1..Bdim,(i,j)-> A[i,j]);
end proc;
```

```
Warning, `delta` is implicitly declared local to procedure `Mat`
Warning, `Baseset` is implicitly declared local to procedure `Mat`
Warning, `ME` is implicitly declared local to procedure `Mat`
Warning, `element` is implicitly declared local to procedure `Mat`
Warning, `i` is implicitly declared local to procedure `Mat`
Warning, `j` is implicitly declared local to procedure `Mat`
Warning, `A` is implicitly declared local to procedure `Mat`
Warning, `Mt` is implicitly declared local to procedure `Mat`
Warning, `si` is implicitly declared local to procedure `Baseset`
Warning, `ei` is implicitly declared local to procedure `Baseset`
Warning, `ci` is implicitly declared local to procedure `Baseset`
Warning, `pri` is implicitly declared local to procedure `Baseset`
```

$$Mat := \mathbf{proc}(NS, NE, kp1, km1, k2) \qquad (3)$$

```
local delta, Baseset, ME, element, i, j, A, Mt;
delta := proc(a, b) piecewise(a = b, 1, 0) end proc;
Baseset := proc(sm, em, cm, pm)
    local si, ei, ci, pri;
    global B, Bdim;
    i := 1;
    for si from 0 to sm do
        for ei from 0 to em do
            for ci from 0 to cm do
                for pri from 0 to pm do
                    if ei + ci = em then
                        if si + ci + pri = sm then B[i] := si, ei, ci, pri; i := i + 1 end if
                    end if
                end do
            end do
        end do
    end do;
    Bdim := i - 1
end proc;
Baseset(NS, NE, NE, NS);
ME := proc(si, sit, ei, eit, ci, cit, pri, prit, kp1, km1, k2)
    ( - delta(ei, eit) * delta(ci, cit) * ((k2 + km1) * cit + kp1 * sit * eit) * delta(si, sit)
     + kp1 * sit * eit * delta(si, sit - 1) * delta(ei, eit - 1) * delta(ci, cit + 1) + km1 * cit
    * delta(si, sit + 1) * delta(ei, eit + 1) * delta(ci, cit - 1)) * delta(pri, prit) + k2 * cit
    * delta(si, sit) * delta(ei, eit + 1) * delta(ci, cit - 1) * delta(pri, prit + 1)
```

```
    end proc;
    element := proc(i, j, kp1, km1, k2)
        ME(B[i][1], B[j][1], B[i][2], B[j][2], B[i][3], B[j][3], B[i][4], B[j][4], kp1, km1,
        k2)
    end proc;
    for i to Bdim do for j to Bdim do A[i,j] := element(i, j, kp1, km1, k2) end do end do;
    Mt := Array(1 ..Bdim, 1 ..Bdim, (i, j) → A[i, j])
end proc
```

```
> #Mat(4,2,kp1,km1,k2);
> ExpHt:= proc(NS,NE,kp1,km1,k2,t)
  Mat(NS,NE,kp1,km1,k2):
  MatrixExponential(Mat(NS,NE,1.0*kp1,1.0*km1,1.0*k2)*t); end proc;
```

```
ExpHt := proc(NS, NE, kp1, km1, k2, t)                                    (4)
    Mat(NS, NE, kp1, km1, k2);
    LinearAlgebra:-MatrixExponential(Mat(NS, NE, 1.0 * kp1, 1.0 * km1, 1.0 * k2) * t)
end proc
```

```
> Expi:=proc(L,NS,NE,kp1,km1,k2,t)
  Mat(NS,NE,kp1,km1,k2):
  evalf(ExpHt(NS,NE,1.0*kp1,1.0*km1,1.0*k2,t)[L][Bdim]);
  end proc;
```

```
Expi := proc(L, NS, NE, kp1, km1, k2, t)                                  (5)
    Mat(NS, NE, kp1, km1, k2); evalf(ExpHt(NS, NE, 1.0 * kp1, 1.0 * km1, 1.0 * k2, t)[L]
    [Bdim])
end proc
```

```
> Vexp:=proc(NS,NE,kp1,km1,k2,t)
  Mat(NS,NE,kp1,km1,k2):
  <seq(Expi(i,NS,NE,kp1,km1,k2,t),i=1..Bdim)>: end proc;
```

```
Vexp := proc(NS, NE, kp1, km1, k2, t)                                     (6)
    Mat(NS, NE, kp1, km1, k2); < seq(Expi(i, NS, NE, kp1, km1, k2, t), i = 1 ..Bdim) >
end proc
```

```
> Vs:=proc(k,NS,NE)
  Mat(NS,NE,kp1,km1,k2):
  <seq(1.0*B[i][1]**k,i=1..Bdim)>: end proc;


  Ve:= proc(k,NS,NE)
  Mat(NS,NE,kp1,km1,k2):
  <seq(1.0*B[i][2]**k,i=1..Bdim)>: end proc;

  Vc:= proc(k,NS,NE)
  Mat(NS,NE,kp1,km1,k2):
  <seq(1.0*B[i][3]**k,i=1..Bdim)>: end proc;
```

```
  Vp:= proc(k,NS,NE)

  Mat(NS,NE,kp1,km1,k2):
  <seq(1.0*B[i][4]**k,i=1..Bdim)>: end proc;
```

$$Vs := \mathbf{proc}(k, NS, NE)\ Mat(NS, NE, kp1, km1, k2);\ <seq(1.0 * B[i][1]\hat{}k, i = 1..Bdim)>\ \mathbf{end\ proc}$$

$$Ve := \mathbf{proc}(k, NS, NE)\ Mat(NS, NE, kp1, km1, k2);\ <seq(1.0 * B[i][2]\hat{}k, i = 1..Bdim)>\ \mathbf{end\ proc}$$

$$Vc := \mathbf{proc}(k, NS, NE)\ Mat(NS, NE, kp1, km1, k2);\ <seq(1.0 * B[i][3]\hat{}k, i = 1..Bdim)>\ \mathbf{end\ proc}$$

$$Vp := \mathbf{proc}(k, NS, NE)\ Mat(NS, NE, kp1, km1, k2);\ <seq(1.0 * B[i][4]\hat{}k, i = 1..Bdim)>\ \mathbf{end\ proc} \quad (7)$$

```
> Sk:=proc(k,NS,NE,kp1,km1,k2,t) DotProduct(Vs(k,NS,NE),Vexp(NS,NE,
  kp1,km1,k2,t)); end proc;
  Ek:=proc(k,NS,NE,kp1,km1,k2,t) DotProduct(Ve(k,NS,NE),Vexp(NS,NE,
  kp1,km1,k2,t)); end proc;
  Ck:=proc(k,NS,NE,kp1,km1,k2,t) DotProduct(Vc(k,NS,NE),Vexp(NS,NE,
  kp1,km1,k2,t)); end proc;
  Pk:=proc(k,NS,NE,kp1,km1,k2,t) DotProduct(Vp(k,NS,NE),Vexp(NS,NE,
  kp1,km1,k2,t)); end proc;
```

$$Sk := \mathbf{proc}(k, NS, NE, kp1, km1, k2, t)\ LinearAlgebra{:}\text{-}DotProduct(Vs(k, NS, NE), Vexp(NS, NE, kp1, km1, k2, t))\ \mathbf{end\ proc}$$

$$Ek := \mathbf{proc}(k, NS, NE, kp1, km1, k2, t)\ LinearAlgebra{:}\text{-}DotProduct(Ve(k, NS, NE), Vexp(NS, NE, kp1, km1, k2, t))\ \mathbf{end\ proc}$$

$$Ck := \mathbf{proc}(k, NS, NE, kp1, km1, k2, t)\ LinearAlgebra{:}\text{-}DotProduct(Vc(k, NS, NE), Vexp(NS, NE, kp1, km1, k2, t))\ \mathbf{end\ proc}$$

$$Pk := \mathbf{proc}(k, NS, NE, kp1, km1, k2, t)\ LinearAlgebra{:}\text{-}DotProduct(Vp(k, NS, NE), Vexp(NS, NE, kp1, km1, k2, t))\ \mathbf{end\ proc} \quad (8)$$

```
> Sp:=pointplot([seq([t,evalf(Sk(1,1,1,100,50,1,t))],t=0..5,0.01)],
  color=magenta,connect=true):
  Ep:=pointplot([seq([t,evalf(Ek(1,1,1,100,50,1,t))],t=0..5,0.01)],
  color=red,connect=true):
  Cp:=pointplot([seq([t,evalf(Ck(1,1,1,100,50,1,t))],t=0..5,0.01)],
  color=blue,connect=true):
  Pp:=pointplot([seq([t,evalf(Pk(1,1,1,100,50,1,t))],t=0..5,0.01)],
  color=green,connect=true):
```

```
display(Sp,Ep,Cp,Pp);
```

